Bazar.com - A Multi-tier Online Bookstore

Muath Hassoun - 12218039
Individual Submission
Lab 1 for Distributed and Operating Systems (Spring 2026)

Bazar.com is a distributed online bookstore implemented as a multi-tier microservices architecture using Node.js and Express. The system consists of three independent services that communicate via HTTP REST APIs: a Catalog service for book inventory management, an Order service for purchase processing, and a Frontend service that acts as the user-facing API gateway. The application demonstrates key distributed systems concepts including service decomposition, inter-service communication, and containerization with Docker.

Architecture Overview

The application follows a 2-tier microservices design:

- **Frontend Service** (Port 3000): User-facing API gateway that routes requests to appropriate backend services
- **Catalog Service** (Port 3001): Manages book inventory, search, and updates
- **Order Service** (Port 3002): Handles purchase transactions and order logging

```
Client → Frontend(:3000) → Catalog(:3001)
                         ↘
                           Order(:3002) → Catalog(:3001)

Client (curl/Postman)
    ↓
Frontend Service (:3000)
    ↓
├── Catalog Service (:3001) ← Order Service (:3002)
│   ├── GET /search/:topic
│   ├── GET /info/:id
│   └── PUT /update/:id
```

```
└── Order Service (:3002)
    └── POST /purchase/:id
```

Services communicate via HTTP REST APIs. The Order service calls the Catalog service to check stock availability and decrement quantities during purchases.

Quick Start

Get the entire system running in 3 commands:

```
cd docker
docker compose up --build
```

Wait for all services to start, then test:

```
curl http://localhost:3000/search/distributed%20systems
curl -X POST http://localhost:3000/purchase/1
```

Project Structure

```
bazar-project/
├── README.md                    # This file - project
documentation
├── catalog/                     # Catalog microservice
│   ├── index.js                 # Main Express server (port 3001)
│   ├── package.json             # Dependencies: express, body-
parser
│   └── routes/
│       ├── query.js             # GET /search/:topic, GET /
info/:id endpoints
│       └── update.js            # PUT /update/:id endpoint
│   └── data/
│       └── catalog.json         # Book inventory data (4 books)
├── order/                       # Order microservice
│   ├── index.js                 # Main Express server (port 3002)
│   ├── package.json             # Dependencies: express, body-
parser, axios
│   └── routes/
│       └── purchase.js          # POST /purchase/:id endpoint
│   └── data/
│       └── orders.json          # Purchase order history
├── frontend/                    # Frontend microservice
│   ├── index.js                 # Main Express server (port 3000)
│   ├── package.json             # Dependencies: express, body-
parser, axios
│   └── routes/
│       ├── search.js            # GET /search/:topic proxy to
```

```
catalog
│       ├── info.js              # GET /info/:id proxy to catalog
│       └── purchase.js          # POST /purchase/:id proxy to
order
└── docker/                      # Docker containerization
    ├── catalog.Dockerfile       # Container recipe for catalog
service
    ├── order.Dockerfile         # Container recipe for order
service
    ├── frontend.Dockerfile      # Container recipe for frontend
service
    └── docker-compose.yml       # Multi-container orchestration
```

File Descriptions

- **`catalog/index.js`**: Express server that sets up middleware and routes for the catalog service
- **`catalog/routes/query.js`**: Handles book search and info retrieval, reads catalog.json fresh each time
- **`catalog/routes/update.js`**: Handles book updates (price/quantity), modifies catalog.json
- **`catalog/data/catalog.json`**: JSON file storing book inventory with id, title, topic, price, quantity
- **`order/index.js`**: Express server for order processing
- **`order/routes/purchase.js`**: Processes purchases, calls catalog for stock check/update, logs to orders.json
- **`order/data/orders.json`**: JSON file storing purchase history with orderId, bookId, title, price, timestamp
- **`frontend/index.js`**: Express server that proxies all requests to backend services
- **`frontend/routes/search.js`**: Proxies search requests to catalog service
- **`frontend/routes/info.js`**: Proxies info requests to catalog service
- **`frontend/routes/purchase.js`**: Proxies purchase requests to order service
- **`docker/*.Dockerfile`**: Docker container recipes using Node.js Alpine images
- **`docker/docker-compose.yml`**: Orchestrates all three services with networking and environment variables

Books Catalog

The system contains 4 books across 2 topics:

ID	Title	Topic	Price	Quantity
1	How to get a good grade in DOS in 40 minutes a day	distributed systems	$40	10
2	RPCs for Noobs	distributed systems	$50	5
3	Xen and the Art of Surviving Undergraduate School	undergraduate school	$35	7
4	Cooking for the Impatient Undergrad	undergraduate school	$30	8

API Reference

Catalog Server (Port 3001)

GET /search/:topic

Returns all books matching the specified topic.

Status Codes: 200 OK, 404 Not Found

Example Request:

```
curl http://localhost:3001/search/distributed%20systems
```

Example Response:


```
[
  {
    "id": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "topic": "distributed systems",
    "price": 40,
    "quantity": 8
  },
  {
    "id": 2,
    "title": "RPCs for Noobs",
    "topic": "distributed systems",
```

```
    "price": 50,
    "quantity": 4
  }
]
```

GET /info/:id

Returns detailed information for a specific book.

Status Codes: 200 OK, 404 Not Found

Example Request:

```
curl http://localhost:3001/info/1
```

Example Response:

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "topic": "distributed systems",
  "price": 40,
  "quantity": 8
}
```

PUT /update/:id

Updates price or quantity for a specific book.

Status Codes: 200 OK, 404 Not Found

Example Request:

```
curl -X PUT http://localhost:3001/update/1 \
  -H "Content-Type: application/json" \
  -d '{"quantity": 9}'
```

Example Response:

```
{
  "message": "Book updated",
  "book": {
    "id": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "topic": "distributed systems",
    "price": 40,
    "quantity": 9
  }
}
```

Order Server (Port 3002)

POST /purchase/:id

Processes a book purchase, checks stock, decrements quantity, and logs the order.

Status Codes: 200 OK, 400 Bad Request (out of stock), 404 Not Found

Example Request:

```
curl -X POST http://localhost:3002/purchase/1
```

Example Response:


```
{
  "message": "Purchase successful",
  "order": {
    "orderId": 6,
    "bookId": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "price": 40,
    "timestamp": "2026-04-18T18:00:00.000Z"
  }
}
```


Frontend Server (Port 3000)

GET /search/:topic

Proxies search requests to the catalog service.

Status Codes: 200 OK, 404 Not Found

Example Request:

```
curl http://localhost:3000/search/distributed%20systems
```

Example Response: Same as catalog service response above.

GET /info/:id

Proxies info requests to the catalog service.

Status Codes: 200 OK, 404 Not Found

Example Request:

```
curl http://localhost:3000/info/1
```

Example Response: Same as catalog service response above.

POST /purchase/:id

Proxies purchase requests to the order service.

Status Codes: 200 OK, 400 Bad Request (out of stock), 404 Not Found

Example Request:

```
curl -X POST http://localhost:3000/purchase/1
```

Example Response: Same as order service response above.

Data Models

Book Object (catalog.json)

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "topic": "distributed systems",
  "price": 40,
  "quantity": 8
}
```

Order Object (orders.json)

```
{
  "orderId": 1,
  "bookId": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "price": 40,
  "timestamp": "2026-04-18T17:11:03.840Z"
}
```

Setup & Installation

Prerequisites

- Node.js v20+ (tested with v20.19.4)
- npm v9+ (tested with v9.2.0)

- Docker v29+ (tested with v29.4.0)

Install Dependencies

```
cd catalog && npm install
cd ../order && npm install
cd ../frontend && npm install
```

Running Without Docker

Open three separate terminals:

Terminal 1 - Catalog Server:

```
cd catalog && npm start
```

Expected output: `Catalog Server running on http://localhost:3001`

Terminal 2 - Order Server:

```
cd order && npm start
```

Expected output: `Order Server running on http://localhost:3002`

Terminal 3 - Frontend Server:

```
cd frontend && npm start
```

Expected output: `Frontend Server running on http://localhost:3000`

Running With Docker

```
cd docker
docker compose up --build
```

Expected output:

```
catalog-service    | Catalog Server running on http://
localhost:3001
order-service      | Order Server running on http://localhost:3002
frontend-service   | Frontend Server running on http://
localhost:3000
```

Testing - Complete Examples with Expected Output

Search by topic:

```
# Search distributed systems books
curl http://localhost:3000/search/distributed%20systems
```

Expected response:


```
[
  {
    "id": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "topic": "distributed systems",
    "price": 40,
    "quantity": 8
  },
  {
    "id": 2,
    "title": "RPCs for Noobs",
    "topic": "distributed systems",
    "price": 50,
    "quantity": 4
  }
]
```


```
# Search undergraduate school books
curl http://localhost:3000/search/undergraduate%20school
```

Expected response:


```
[
  {
    "id": 3,
    "title": "Xen and the Art of Surviving Undergraduate School",
    "topic": "undergraduate school",
    "price": 35,
    "quantity": 6
  },
  {
    "id": 4,
    "title": "Cooking for the Impatient Undergrad",
    "topic": "undergraduate school",
    "price": 30,
    "quantity": 8
  }
]
```

Get book info:

```
curl http://localhost:3000/info/1
```

Expected response:

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "topic": "distributed systems",
  "price": 40,
  "quantity": 8
}
```

Purchase a book:

```
curl -X POST http://localhost:3000/purchase/1
```

Expected response:

```
{
  "message": "Purchase successful",
  "order": {
    "orderId": 6,
    "bookId": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "price": 40,
    "timestamp": "2026-04-18T18:00:00.000Z"
  }
}
```

Verify quantity decreased:

```
curl http://localhost:3000/info/1
```

Expected response (quantity now 7):

```
{
  "id": 1,
  "title": "How to get a good grade in DOS in 40 minutes a day",
  "topic": "distributed systems",
  "price": 40,
  "quantity": 7
}
```

Out of stock test:

Purchase book 2 multiple times until quantity reaches 0, then try again:

```
curl -X POST http://localhost:3000/purchase/2
# Repeat until quantity = 0
curl -X POST http://localhost:3000/purchase/2
```

Expected response when out of stock:

```
{
  "error": "Book out of stock"
}
```

Direct catalog calls:

```
# Direct search
curl http://localhost:3001/search/distributed%20systems

# Direct info
curl http://localhost:3001/info/1

# Direct update
curl -X PUT http://localhost:3001/update/1 \
  -H "Content-Type: application/json" \
  -d '{"quantity": 15}'
```

Request Flow - How It Works

Search Flow

1. Client sends `GET /search/:topic` to Frontend (port 3000)
2. Frontend proxies request to Catalog service `GET /search/:topic`
3. Catalog reads `catalog.json`, filters books by topic, returns JSON array

Info Flow

1. Client sends `GET /info/:id` to Frontend (port 3000)
2. Frontend proxies request to Catalog service `GET /info/:id`
3. Catalog reads `catalog.json`, finds book by ID, returns JSON object

Purchase Flow

1. Client sends `POST /purchase/:id` to Frontend (port 3000)
2. Frontend proxies request to Order service `POST /purchase/:id`
3. Order service calls Catalog `GET /info/:id` to check stock
4. If quantity > 0, Order calls Catalog `PUT /update/:id` with { quantity: currentQuantity - 1 } to decrement quantity by 1

5. Order logs purchase to `orders.json` with orderId, bookId, title, price, timestamp
6. Order returns success response with order details

Design Decisions & Tradeoffs

- **Node.js** + **Express**: Chosen for simplicity, async/await support, and JSON handling. Express handles concurrency automatically with its event loop.
- **JSON Files for Persistence**: No database needed for this lab. Simple file I/O demonstrates distributed data management. Tradeoff: not suitable for high concurrency or large datasets.
- **Microservices Architecture**: Services are independent and can be scaled/deployed separately. Tradeoff: increased complexity in inter-service communication.
- **HTTP REST APIs**: Simple, stateless communication between services. Tradeoff: higher latency compared to direct function calls.
- **Environment Variables**: URLs configured via env vars for flexibility between local development and Docker deployment.
- **No Authentication**: Not required for this lab, keeping focus on distributed systems concepts.

Known Issues & Limitations

- **Concurrency Control**: The file-based storage system lacks atomic transactions, potentially allowing race conditions during concurrent purchase operations that could result in overselling inventory. While Node.js processes requests sequentially within a single instance, distributed deployments would require proper locking mechanisms or database transactions.

- **Fault Tolerance**: Inter-service communication lacks retry logic and circuit breaker patterns. Service failures during purchase operations result in immediate transaction failure without recovery mechanisms, compromising system reliability in production environments.

- **Scalability Constraints**: JSON file persistence is unsuitable for high-throughput scenarios, with potential performance degradation under concurrent read/write operations and lack of data persistence guarantees across container restarts.

- **Input Validation**: API endpoints implement minimal parameter validation, potentially vulnerable to malformed requests and injection attacks in production deployments.

Possible Improvements & Extensions

- **Database Integration**: Replace JSON files with SQLite/PostgreSQL for ACID transactions and better concurrency control.
- **Caching Layer**: Add Redis cache for frequently accessed book data to reduce catalog service load.
- **Load Balancing**: Deploy multiple instances of each service behind a load balancer.
- **Authentication & Authorization**: Add user accounts, JWT tokens, and role-based access control.
- **API Gateway**: Implement proper API gateway (e.g., Kong, Express Gateway) for routing, rate limiting, and monitoring.
- **Monitoring & Logging**: Add centralized logging (ELK stack) and metrics collection (Prometheus).
- **Circuit Breaker**: Implement circuit breaker pattern for resilient inter-service communication.
- **Event-Driven Architecture**: Use message queues (RabbitMQ, Kafka) for asynchronous order processing.
- **Container Orchestration**: Deploy on Kubernetes for production-grade container management.
- **Testing**: Add unit tests (Jest), integration tests, and end-to-end tests with test containers.